Rate limiter design in Go

Contents

1. Why rate limit
2. Where to apply it
3. The four classic algorithms
4. Algorithm comparison
5. golang.org/x/time/rate (token bucket)
6. uber-go/ratelimit (leaky bucket)
7. HTTP middleware: per-IP
8. Distributed rate limiting with Redis
9. Library choices
10. Design choices
11. Edge cases and gotchas
12. Worked example: complete HTTP middleware
13. Observability
14. Testing rate limiters
15. Best practices
16. Common gotchas
17. Quick reference

How to limit the rate of operations: the algorithms, the Go libraries, the distributed variants, and the design choices that bite in production.

1. Why rate limit

- **Protect resources.** A burst of traffic from one client shouldn't starve everyone else.
- **Enforce quotas.** Different tiers (free, pro, enterprise) get different allowances.
- **Defend against abuse.** Brute-force login, scraping, DDoS amplification.
- **Smooth bursty traffic.** Downstream systems get a predictable input rate.
- **Bound cost.** Calls to paid third-party APIs (OpenAI, Stripe) need a cap.

Rate limiting is not the same as throttling or back-pressure:

Term	Meaning
Rate limit	Hard cap; reject above the limit
Throttle	Soft cap; queue or slow down above the limit

Term	Meaning
Back-pressure	Signal upstream to slow down (queues fill, latency rises)
Circuit breaker	Skip an unhealthy downstream entirely

A well-designed system uses all four together.

2. Where to apply it

Layer	What you can limit by	Example
CDN / edge	IP, region, headers	Cloudflare WAF
API gateway	API key, user ID, route	Kong, Envoy
Load balancer	IP, hostname	Nginx `limit_req`
Service (your code)	User, tenant, account, endpoint, IP	This sheet
Database / cache	Concurrent connections	Connection pool size

Limit at the outermost layer that has enough information. Edge for IPs, gateway for API keys, your service for business logic (per-user rate, per-tenant quota).

3. The four classic algorithms

Token bucket

A bucket holds up to `B` tokens. Tokens refill at rate `R` per second. Each request consumes one token. If the bucket is empty, the request is denied (or queued).

```
capacity: B (burst size)
refill:   R tokens / sec
on request:
    refill bucket based on elapsed time, capped at B
    if tokens >= 1:
        tokens -= 1; allow
    else:
        deny
```

Properties:

- Allows bursts up to `B` requests when the bucket is full.
- Long-term average is `R` requests/second.
- Self-stabilizing: a quiet period accumulates tokens.

Standard choice. Used by `golang.org/x/time/rate`, AWS, GitHub.

Leaky bucket

A queue drains at fixed rate `R`. New requests are added if the queue has space; rejected if full.

```
capacity: B (queue size)
drain:    R requests / sec
on request:
    if queue has space: enqueue
    else: deny
output: process queue at rate R
```

Properties:

- Output is smooth at exactly `R`. No bursts at the consumer.
- Adds latency: requests wait in the queue.
- Doesn't allow bursts; same rate every time.

Use when downstream needs strictly smoothed input (e.g., a fragile legacy system).

Fixed window counter

Count requests within a fixed window (e.g., every 60 seconds). Reject above the limit. Reset at window boundary.

```
window: 60 sec
limit: 100 per window
on request:
    bucket = current_window
    count[bucket]++
    if count[bucket] > limit: deny
```

Properties:

- Cheap: one counter per window per key.
- Edge problem: 100 requests at 0:59 and another 100 at 1:00 looks like a single burst of 200 to anyone observing.

Common because it's easy. Acceptable when the boundary spike doesn't matter.

Sliding window log

Store the timestamp of every request in the last window. On each request, drop stamps older than the window and count what remains.

```
window: 60 sec
limit: 100
on request:
    drop timestamps older than now − 60 sec
    if remaining count + 1 > limit: deny
    else: append now
```

Properties:

- Exact: no boundary problem.
- Memory: one entry per request per key (expensive for high QPS).

Used in low-volume cases. Rarely the right call at scale.

Sliding window counter

A hybrid. Keep counters for the current and previous fixed window. Weight the previous window's count by the fraction still inside the sliding range.

```
window: 60 sec
limit: 100
on request:
    current = count in current 60-sec window
    previous = count in previous 60-sec window
    elapsed_in_current = (now − window_start) / 60
    weighted = previous ∗ (1 − elapsed_in_current) + current
    if weighted + 1 > limit: deny
    else: increment current
```

Properties:

- Close to exact (within a few percent).
- Cheap: two counters per key.
- No boundary spike.

The pragmatic production choice for sliding limits.

4. Algorithm comparison

Algorithm	Burst	Smoothing	Memory	Boundary problem	Use case
Token bucket	Yes (up to B)	Soft	O(1) per key	None	General-purpose, default
Leaky bucket	No	Hard	O(B) per key	None	Strict-rate downstream

Algorithm	Burst	Smoothing	Memory	Boundary problem	Use case
Fixed window	At boundary	Window-level	O(1) per key	Yes (2x spike)	Cheap and rough
Sliding window log	No spike	Exact	O(N) per key	None	Low volume, exact
Sliding window counter	Smooth	Approximate	O(1) per key	Near-none	Production sliding limits

5. golang.org/x/time/rate (token bucket)

The standard token-bucket implementation. In-process, well-tested.

```
import "golang.org/x/time/rate"

// 10 events per second, with a burst of 20
limiter := rate.NewLimiter(rate.Limit(10), 20)
```

Allow / Wait / Reserve

```
// non-blocking: returns true if allowed, false otherwise
if !limiter.Allow() {
    http.Error(w, "rate limited", http.StatusTooManyRequests)
    return
}

// blocking: waits until a token is available (or ctx cancels)
if err := limiter.Wait(ctx); err != nil {
    return err
}

// reserve a token; returns a Reservation with a delay
r := limiter.Reserve()
if !r.OK() {
    return errors.New("would block too long")
}
time.Sleep(r.Delay())
// or r.Cancel() if you decide not to proceed
```

Allow N at once

```
limiter.AllowN(time.Now(), 5)          // ask for 5 tokens
limiter.WaitN(ctx, 5)
```

Per-key limiter

For limiting per-user, you need a map of limiters:

```
type Limiters struct {
    mu sync.Mutex
    lim map[string]*rate.Limiter
}

func (l *Limiters) get(key string) *rate.Limiter {
    l.mu.Lock()
    defer l.mu.Unlock()
    if v, ok := l.lim[key]; ok { return v }
    v := rate.NewLimiter(10, 20)
    l.lim[key] = v
    return v
}
```

Add cleanup so the map doesn't grow forever: evict entries with no recent activity. An LRU cache with a TTL works well.

Limitations

- Local to one process. If you have 5 replicas with limit 10/sec each, your total system rate is 50/sec.
- Memory grows with key cardinality. Bounded eviction is essential.

For multi-instance services, you need a distributed limiter (see below).

6. uber-go/ratelimit (leaky bucket)

Different model: callers block on `Take()` and proceed at fixed rate.

```
import "go.uber.org/ratelimit"

rl := ratelimit.New(100)          // 100 requests per second

for i := 0; i < 1000; i++ {
    rl.Take()              // blocks to enforce rate
    doWork()
}
```

Use when you're the producer of calls and want to space them out, e.g., a worker that calls a paid API. Different from `x/time/rate`, which is for gatekeeping incoming traffic.

7. HTTP middleware: per-IP

```go
func rateLimitByIP(next http.Handler) http.Handler {
    limiters := newLimiterStore(rate.Limit(10), 20, time.Hour)

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        ip := clientIP(r)
        lim := limiters.get(ip)

        if !lim.Allow() {
            w.Header().Set("Retry-After", "1")
            http.Error(w, "rate limited", http.StatusTooManyRequests)
            return
        }
        next.ServeHTTP(w, r)
    })
}

func clientIP(r *http.Request) string {
    if v := r.Header.Get("X-Forwarded-For"); v != "" {
        return strings.Split(v, ",")[0]
    }
    return strings.Split(r.RemoteAddr, ":")[0]
}
```

`X-Forwarded-For` matters when behind a proxy. Trust it only when you control the proxy chain; otherwise it's spoofable.

Standard response headers

When rate limited:

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1716559200
```

`Retry-After` is the official header (seconds or HTTP date). The `X-RateLimit-*` headers are convention; GitHub popularized them.

When succeeding, include the headers too, so well-behaved clients can self-throttle.

8. Distributed rate limiting with Redis

For multiple service instances sharing one limit, Redis is the standard backend. The limiter state lives there.

Fixed window in Redis

```
func allow(ctx context.Context, rdb *redis.Client, key string, limit int, window
        time.Duration) (bool, error) {
    bucket := time.Now().Unix() / int64(window.Seconds())
    rkey := fmt.Sprintf("rl:%s:%d", key, bucket)

    n, err := rdb.Incr(ctx, rkey).Result()
    if err != nil { return false, err }
    if n == 1 {
        rdb.Expire(ctx, rkey, window+time.Second)
    }
    return n <= int64(limit), nil
}
```

Cheap. Has the boundary problem.

Sliding window log with sorted sets

```
// store one entry per request; remove old ones; count
script := redis.NewScript(`
    local key = KEYS[1]
    local now = tonumber(ARGV[1])
    local window = tonumber(ARGV[2])
    local limit = tonumber(ARGV[3])

    redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
    local count = redis.call('ZCARD', key)
    if count + 1 > limit then
        return 0
    end
    redis.call('ZADD', key, now, now)
    redis.call('PEXPIRE', key, window)
    return 1
`)

ok, err := script.Run(ctx, rdb, []string{"rl:" + key},
    time.Now().UnixMilli(), windowMs, limit).Int()
```

Exact. Higher memory; one ZSET entry per request.

Token bucket in Redis

Atomic via Lua. Stores tokens and `last_refill` per key.

```lua
local key = KEYS[1]
local capacity = tonumber(ARGV[1])
local refill_rate = tonumber(ARGV[2])                -- tokens per second
local now = tonumber(ARGV[3])                        -- ms
local cost = tonumber(ARGV[4])

local data = redis.call('HMGET', key, 'tokens', 'last')
local tokens = tonumber(data[1]) or capacity
local last = tonumber(data[2]) or now

local elapsed = (now - last) / 1000
tokens = math.min(capacity, tokens + elapsed * refill_rate)

local allowed = 0
if tokens >= cost then
    tokens = tokens - cost
    allowed = 1
end

redis.call('HMSET', key, 'tokens', tokens, 'last', now)
redis.call('PEXPIRE', key, math.ceil(capacity / refill_rate * 1000) + 1000)

return allowed
```

The Lua script ensures atomicity. Without it, two concurrent updates can over-grant.

Why Lua

Redis runs Lua scripts atomically. `INCR` is also atomic, but multi-step logic (check + update) needs Lua to prevent races between multiple instances.

Cost: one round trip per request

Every limited request pays a Redis call. At 10ms p99 to Redis, that's the floor for your endpoint latency. Mitigations:

- Co-locate Redis. Same datacenter, same network zone.
- Batch. If multiple operations share a key, combine them.
- Local cache + periodic sync. See below.

Fail open vs fail closed

What if Redis is down?

- **Fail open:** allow everything when the limiter can't be checked. Safer for availability; risk of overload.
- **Fail closed:** deny everything. Safer for the resource; bad UX when Redis blips.

Default to fail open for normal user traffic, fail closed for security-critical limits (login attempts, payment).

Hybrid: local + distributed

Pure distributed is expensive. Pure local doesn't scale. The hybrid: each instance has a local token bucket, periodically syncs with Redis to share the global budget.

Approaches:

- **Lease tokens from Redis in chunks.** Instance pulls 10 tokens at a time and serves them locally.
- **Periodic reconciliation.** Each instance reports usage every second; receives a refreshed allowance.

Tradeoff: occasional over-grant during sync windows. Acceptable for most cases.

`github.com/throttled/throttled/v2` and `github.com/sethvargo/go-limiter` implement variants.

9. Library choices

Library	Algorithm	Distributed?	Notes
`golang.org/x/time/rate`	Token bucket	No	Stdlib-grade, default for in-process
`go.uber.org/ratelimit`	Leaky bucket	No	Blocking caller model
`github.com/throttled/throttled`	Multiple	Yes (Redis, memstore)	HTTP middleware focus
`github.com/sethvargo/go-limiter`	Multiple	Yes (Redis, mem, noop)	Pluggable backends
`github.com/ulule/limiter`	Token bucket	Yes (Redis, memcached)	Easy HTTP middleware
`github.com/go-redis/redis_rate/v10`	GCRA	Yes (Redis only)	Compact, used by go-redis

`redis_rate` uses the GCRA algorithm (Generic Cell Rate Algorithm), a smooth variant of token bucket commonly used in network gear.

10. Design choices

What's the key?

- **IP:** simplest, but unreliable (NAT, proxies, mobile networks share IPs).
- **User ID:** good for authenticated traffic.
- **Tenant / account ID:** for SaaS, the natural billing/quota boundary.
- **API key:** for B2B APIs.
- **API key + endpoint:** different limits per route.
- **API key + status (success vs error):** prevent retries from gaming.

You can compose: `userID:endpoint` for “100 calls to /search per user per minute.”

Key cardinality drives memory. A limit per IP+endpoint with millions of IPs and hundreds of endpoints is hundreds of millions of buckets.

Burst size

Token bucket capacity. Tuning:

- Too small: legitimate bursts (user opens 5 tabs) get rejected.
- Too large: an attacker drains the entire budget at once, then waits.

> **Rule of thumb**
>
> `burst = rate * window`, where window is “reasonable burst duration” (often 2-5 seconds).

Limit hierarchies

Different limits at different scopes:

```
global: 10,000 req/sec
per-tenant: 1,000 req/sec
per-user: 100 req/sec
per-endpoint: configurable per route
```

Check from cheapest to most expensive. Reject early if any limit fails.

Tier-aware limits

```
limit := 100
if user.Plan == "pro" { limit = 1000 }
if user.Plan == "enterprise" { limit = 10000 }
```

Store plan in user record (cached); read on each request.

Quotas vs rates

	Rate limit	Quota
Window	Short (seconds, minutes)	Long (day, month)
Storage	In-memory OK	Persistent (DB or Redis with persistence)
Reset	Continuous (refill)	Calendar-based
Use	Prevent overload	Billing, monthly allowance

The same algorithms apply; the difference is the window length and the storage backend.

11. Edge cases and gotchas

The thundering herd at reset

Fixed window: all clients hit the limit, all wait, all retry at the boundary. Spike. Use sliding window or randomize the reset:

```
windowStart := userID % windowSize            // per-user offset
```

IP spoofing

`X-Forwarded-For` is a header; anyone can set it. Trust only the addresses your proxy adds. Configure your proxy to strip incoming `X-Forwarded-For` and write its own.

IPv6 cardinality

Each IPv6 address is its own bucket. An attacker with `/48` has 2^80 addresses; per-IP rate limiting is useless against them. Rate limit by `/64` prefix (the smallest reasonable unit).

Retries

A client that retries on 429 can amplify load. Always include `Retry-After`. Honor it client-side.

Synchronous calls in the hot path

A Redis call per request adds latency to every request. If your endpoint is 5ms and Redis is 2ms, you've added 40%. Co-locate, or use a hybrid local+remote.

Counter inflation under retries

Some implementations count failed requests against the limit. Decide: retries due to upstream errors shouldn't count against the user's quota. Either don't count 5xx, or refund the token on error.

Sliding window log memory

At 1000 req/sec per key, the log has 60,000 entries per 60-sec window. Million users = 60 billion entries. Not feasible. Use sliding window counter instead.

Time skew

In distributed limiters, multiple instances must agree on "now." Use Redis's `TIME` command to get the server's time, not the client's clock.

12. Worked example: complete HTTP middleware

```
package middleware

import (
    "context"
    "encoding/json"
    "errors"
    "net/http"
    "strconv"
    "time"

    "github.com/redis/go-redis/v9"
)

type RateLimiter struct {
    rdb       *redis.Client
    limit     int
    window    time.Duration
    keyFunc   func(*http.Request) string
}

func NewRateLimiter(rdb *redis.Client, limit int, window time.Duration, keyFunc
        func(*http.Request) string) *RateLimiter {
    return &RateLimiter{rdb: rdb, limit: limit, window: window, keyFunc: keyFunc}
}

var script = redis.NewScript(`
    local key = KEYS[1]
    local limit = tonumber(ARGV[1])
    local window_ms = tonumber(ARGV[2])
    local now = tonumber(ARGV[3])

    redis.call('ZREMRANGEBYSCORE', key, 0, now - window_ms)
    local count = redis.call('ZCARD', key)
    if count >= limit then
       return {0, count, redis.call('ZRANGE', key, 0, 0, 'WITHSCORES')[2] or '0'}
    end
    redis.call('ZADD', key, now, now .. '-' .. math.random())
    redis.call('PEXPIRE', key, window_ms)
    return {1, count + 1, '0'}
`)

func (rl *RateLimiter) Allow(ctx context.Context, key string) (bool, int,
        time.Duration, error) {
    res, err := script.Run(ctx, rl.rdb,
        []string{"rl:" + key},
        rl.limit, rl.window.Milliseconds(), time.Now().UnixMilli(),
    ).Result()

    if err != nil {
        return true, 0, 0, err   // fail open
    }
    arr := res.([]any)
```

```
    allowed := arr[0].(int64) == 1
    count := int(arr[1].(int64))

    var resetIn time.Duration
    if s, _ := arr[2].(string); s != "" && s != "0" {
        oldest, _ := strconv.ParseInt(s, 10, 64)
        resetIn = time.Until(time.UnixMilli(oldest).Add(rl.window))
    }

    return allowed, count, resetIn, nil
}

func (rl *RateLimiter) Middleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        key := rl.keyFunc(r)
        allowed, count, resetIn, err := rl.Allow(r.Context(), key)

        w.Header().Set("X-RateLimit-Limit", strconv.Itoa(rl.limit))
        w.Header().Set("X-RateLimit-Remaining", strconv.Itoa(rl.limit-count))
        if resetIn > 0 {
            w.Header().Set("X-RateLimit-Reset",
        strconv.FormatInt(time.Now().Add(resetIn).Unix(), 10))
        }

        if err != nil {
            // log and fail open
            next.ServeHTTP(w, r)
            return
        }

        if !allowed {
            w.Header().Set("Retry-After", strconv.Itoa(int(resetIn.Seconds())+1))
            w.WriteHeader(http.StatusTooManyRequests)
            json.NewEncoder(w).Encode(map[string]string{"error": "rate limit
        exceeded"})
            return
        }
        next.ServeHTTP(w, r)
    })
}
```

Usage:

```
limiter := NewRateLimiter(rdb, 100, time.Minute, func(r *http.Request) string {
    return userFromRequest(r)
})

mux := http.NewServeMux()
mux.Handle("/api/", limiter.Middleware(apiHandler))
```

13. Observability

Metrics to emit:

- Allowed count (per limit key, per route).
- Denied count (the metric that justifies the limit).
- Limiter latency (Redis round-trip).
- Limiter errors (Redis down, script failures).
- Quota usage histogram (how close are users to their limit?).

Log denials with key + path so you can spot patterns: one IP scraping, one user with a buggy retry loop, one tenant outgrowing their tier.

Trace span on the limiter check, with the key as an attribute.

14. Testing rate limiters

```go
func TestLimiter(t *testing.T) {
    rl := rate.NewLimiter(10, 20)

    // Burst: first 20 should pass instantly
    for i := 0; i < 20; i++ {
        if !rl.Allow() { t.Fatal("expected allow") }
    }
    // 21st should fail (or wait)
    if rl.Allow() { t.Fatal("expected deny") }

    // After 1 second, we have ~10 more tokens
    time.Sleep(time.Second)
    pass := 0
    for i := 0; i < 15; i++ {
        if rl.Allow() { pass++ }
    }
    if pass < 9 || pass > 11 { t.Fatalf("expected ~10, got %d", pass) }
}
```

For deterministic tests, swap `time.Now` for an injectable clock so tests don't rely on real time.

15. Best practices

1. Start with `golang.org/x/time/rate` for per-instance limits. Solid, simple, stdlib-grade.
2. Go distributed only when you must. Each Redis hop costs latency.
3. Always return `Retry-After` on 429. Clients can self-throttle.
4. Emit `X-RateLimit-*` headers on success too. Helps well-behaved clients.
5. Limit at the most informative key. User ID beats IP for authenticated traffic.

6. Tier-aware: read user plan, apply different limits, cache the lookup.
7. Fail open by default, except for security-sensitive endpoints (login, payment).
8. Bound the limiter store memory. Evict idle keys.
9. Log and alert on high denial rates. Often signals an attack, a bug, or a wrong limit.
10. Test with synthetic load before shipping. A misconfigured limiter is a self-DOS.

16. Common gotchas

Gotcha	Symptom	Fix
Per-instance limit, N instances	Real limit is N x configured	Distributed limiter or hybrid
Unbounded limiter map	Memory leak	LRU + TTL on inactive keys
Trusting `X-Forwarded-For` from anywhere	Spoofable; everyone gets same bucket	Trust only known proxy IPs
IPv6 single-address limit	2^80 addresses per attacker	Limit by `/64` prefix
No `Retry-After` on 429	Clients hammer with retries	Always set
Sliding window log at high QPS	Redis memory explodes	Switch to sliding counter
Redis down, fail closed everywhere	Outage	Fail open with monitoring
Limiter latency added to every request	p99 degraded	Co-locate Redis, use hybrid
Race in custom limiter	Off-by-some allowance	Use Lua for atomic Redis ops
Hot key on Redis (one popular tenant)	Single-shard bottleneck	Pre-shard the key (`key:%shard`)
Counting retries against the limit	Legitimate retry loops fail	Only count user-initiated requests
Clock skew between instances	Inconsistent limit windows	Use Redis `TIME` for the timestamp
Wrong burst size for token bucket	Either over-permissive or rejects normal bursts	Tune based on real traffic distribution
No metric for denials	Don't see when limit is wrong	Counter by route + reason

17. Quick reference

What	Tool / Pattern
In-process token bucket	`golang.org/x/time/rate.NewLimiter(r, b)`
Non-blocking check	`limiter.Allow()`
Blocking wait	`limiter.Wait(ctx)`
Reserve for the future	`limiter.Reserve()`

What	Tool / Pattern
In-process leaky bucket	`go.uber.org/ratelimit`
HTTP middleware (simple)	wrap with `func(http.Handler) http.Handler`
Distributed (Redis simple)	`INCR` + `EXPIRE` per fixed window
Distributed (Redis exact)	Lua sliding window log on sorted sets
Distributed (Redis token bucket)	Lua + HASH for tokens + last_refill
HTTP status when limited	429 Too Many Requests
Required header	`Retry-After`
Convention headers	`X-RateLimit-Limit` / `Remaining` / `Reset`
Library: Redis-backed token bucket	`github.com/go-redis/redis_rate/v10`
Library: pluggable backends	`github.com/sethvargo/go-limiter`
Library: HTTP middleware	`github.com/ulule/limiter`
Per-IP key	`clientIP(r)` with proxy-aware extraction
Per-user key	from auth context
Per-tenant key	from API key lookup
Tier-aware	conditional limit based on user plan
Failure mode	fail open for UX, closed for security
Memory management	LRU + TTL on key map
Observability	metrics: allowed, denied, latency, error